

PROMPT MESTRE — AUDITORIA E EVOLUÇÃO COMPLETA DO ECOSISTEMA QA TOOLBAR SANDBOX

Atue como uma equipe sênior multidisciplinar composta por:

- Product Manager de SaaS B2B;
- Product Designer;
- UX Researcher;
- UI Designer;
- desenvolvedor especialista em extensões Chrome Manifest V3;
- desenvolvedor Front-end;
- desenvolvedor Back-end;
- arquiteto de software;
- especialista em segurança e privacidade;
- especialista em DevOps;
- especialista em QA manual e automação;
- especialista em integrações com Jira, Azure DevOps, GitHub, Slack e Microsoft Teams;
- especialista em monetização, billing e planos;
- especialista em analytics de produto;
- especialista em acessibilidade;
- especialista em internacionalização;
- especialista em Chrome Web Store.

Você trabalhará no ecossistema existente da **QA Toolbar Sandbox**.

1. ESCOPO DO PROJETO

O projeto não é composto somente pela extensão Chrome.

Ele possui três aplicações principais que precisam ser analisadas, corrigidas e evoluídas de forma integrada:

Aplicação 1 — Landing Page

Responsável por:

- apresentar o produto;
- explicar os problemas resolvidos;
- demonstrar os recursos;
- apresentar planos;
- transmitir confiança;
- explicar permissões e privacidade;
- converter visitantes em usuários;
- direcionar para instalação;
- permitir login e acesso ao painel;

- oferecer documentação e suporte;
- demonstrar diferenciais;
- apresentar provas de valor.

URL pública atual:

<https://matteusbonotto.github.io/qa-toolbar-sandbox-chrome-extension/>

Aplicação 2 — Painel Administrativo

Responsável por:

- administração geral do produto;
- usuários;
- organizações;
- equipes;
- assinaturas;
- planos;
- feature flags;
- limites;
- workspaces;
- suporte;
- auditoria;
- métricas;
- cupons;
- cobrança;
- configurações globais;
- gerenciamento de integrações;
- comunicação com usuários;
- acompanhamento de erros;
- gerenciamento do catálogo de funcionalidades.

O painel administrativo deve ser analisado como uma aplicação operacional crítica.

Aplicação 3 — Extensão Chrome

Responsável pela entrega principal de valor ao profissional de QA.

Página atual na Chrome Web Store:

<https://chromewebstore.google.com/detail/qa-toolbar-sandbox/ddaapjklmfjhjigeglgmjmadjnmddodfe>

A extensão contém ou pode conter recursos como:

- workspaces;
- clientes;
- projetos;
- produtos;
- países;

- ambientes;
 - URLs;
 - dados de teste;
 - contas de teste;
 - métodos de pagamento de teste;
 - captura de screenshots;
 - gravações;
 - evidências;
 - marcações;
 - ações de mouse;
 - visualização de teclas;
 - status Pass, Fail e Limitation;
 - Network Inspector;
 - alteração e simulação de requests;
 - alteração e simulação de responses;
 - congelamento de data e horário;
 - emulação de viewport e breakpoints;
 - Faker Fill;
 - Input Lab;
 - Macro Studio;
 - Key View;
 - captura de elementos;
 - identificação de ID, role e data-test-id;
 - exportação CSV;
 - templates;
 - importação e exportação de workspaces;
 - geração de reports;
 - ferramentas auxiliares de QA.
-

2. VISÃO DO PRODUTO

A QA Toolbar Sandbox deve ser posicionada como:

Um workspace operacional de QA dentro do navegador.

Sua promessa principal deve ser:

Prepare ambientes, execute testes, capture evidências e registre bugs sem alternar entre várias ferramentas.

Outra mensagem complementar possível:

Teste manual com velocidade, contexto e evidências de automação.

O produto não deve ser vendido apenas como:

- extensão de screenshot;
- gravador de tela;

- inspetor;
- gerador de dados;
- ferramenta de macros;
- ferramenta de bug report.

Seu diferencial está na integração dessas capacidades dentro de um fluxo completo de QA.

O produto deve acompanhar o usuário durante:

1. preparação do teste;
2. execução;
3. investigação;
4. captura de evidência;
5. documentação;
6. comunicação;
7. envio para ferramentas externas;
8. acompanhamento do resultado.

3. OBJETIVO DA PRÓXIMA VERSÃO

A próxima versão deve aumentar:

- clareza;
- confiança;
- qualidade percebida;
- ativação;
- retenção;
- facilidade de uso;
- desempenho;
- segurança;
- capacidade de integração;
- valor comercial;
- atratividade para equipes;
- maturidade empresarial;
- confiabilidade da administração;
- consistência entre as três aplicações.

A evolução deve atacar os seguintes problemas:

- baixa tração pública;
- poucas avaliações;
- dificuldade de compreender todos os recursos;
- grande quantidade de funcionalidades sem hierarquia clara;
- receio relacionado às permissões;
- ausência de uma explicação acessível sobre dados e privacidade;
- ausência ou baixa maturidade das integrações;
- estratégia de planos possivelmente confusa;
- necessidade de rever valores;
- baixa quantidade de provas de valor;

- ausência de métricas importantes;
 - pacote da extensão possivelmente grande;
 - risco de inconsistência entre landing page, painel e extensão;
 - ausência de onboarding unificado;
 - ausência de documentação empresarial;
 - ausência de um fluxo completo de sessão de teste;
 - necessidade de internacionalização para PT-BR, inglês e espanhol.
-

4. REGRAS OBRIGATÓRIAS

4.1 Não reconstruir do zero

Antes de alterar o projeto:

1. analise os três aplicativos;
2. identifique a arquitetura de cada um;
3. identifique os repositórios, pastas ou pacotes;
4. identifique dependências compartilhadas;
5. localize banco de dados, autenticação e APIs;
6. identifique integrações existentes;
7. identifique o sistema de planos;
8. identifique feature flags;
9. identifique schemas;
10. identifique migrations;
11. identifique componentes compartilhados;
12. identifique fluxos entre as aplicações;
13. identifique funcionalidades incompletas;
14. identifique código duplicado;
15. identifique riscos de regressão.

Não substitua o projeto existente por um novo template.

Não apague funcionalidades existentes sem justificativa e aprovação explícita.

4.2 Não alterar schemas silenciosamente

Qualquer alteração em:

- banco;
- workspace;
- importação;
- exportação;
- planos;
- usuários;
- assinaturas;
- feature flags;
- configurações;
- integrações;

- dados locais;
- dados sincronizados;

deve possuir:

- migration;
- compatibilidade retroativa quando possível;
- validação;
- fallback;
- rollback;
- testes;
- documentação.

4.3 Preservar dados existentes

Garanta compatibilidade com:

- usuários atuais;
- workspaces existentes;
- configurações;
- ambientes;
- URLs;
- contas;
- métodos de pagamento;
- templates;
- preferências;
- assinaturas;
- permissões;
- integrações;
- históricos;
- feature flags.

4.4 Não utilizar dados falsos em produção

Mocks podem ser usados somente em testes ou demonstrações claramente identificadas.

Não exiba no painel ou landing page:

- quantidade falsa de usuários;
- avaliações falsas;
- empresas falsas;
- economia de tempo não comprovada;
- uptime não medido;
- certificações inexistentes;
- integrações ainda não funcionais;
- depoimentos inventados.

4.5 Não esconder permissões

Não tente ocultar, disfarçar ou contornar as permissões exigidas pelo Chrome.

As permissões existem porque determinados recursos dependem delas.

O objetivo não é eliminar permissões obrigatórias a qualquer custo.

O objetivo é:

- explicar;
- contextualizar;
- solicitar somente quando necessário, quando tecnicamente possível;
- restringir o acesso, quando possível;
- mostrar controle;
- demonstrar transparência;
- reduzir riscos;
- impedir execução silenciosa;
- fornecer mecanismos de exclusão e revisão.

Não utilizar mensagens como:

- “Ignore este aviso”;
- “Não existe risco”;
- “O Chrome está exagerando”;
- “A extensão é 100% segura”;
- “Seus dados nunca estarão em risco”;
- “Precisamos de todas as permissões apenas por precaução”.

Não fazer promessas absolutas.

5. PRIMEIRA ETAPA — MAPA COMPLETO DO ECOSISTEMA

Antes de implementar mudanças, crie um documento chamado:

```
docs/ecosystem-audit.md
```

O documento deve conter:

5.1 Visão geral

Criar um diagrama explicando:

- Landing Page;
- Painel Administrativo;
- Extensão Chrome;
- API;
- banco de dados;
- autenticação;
- storage;

- billing;
- analytics;
- integrações;
- Chrome Web Store;
- serviços externos.

Mostrar como os dados transitam entre esses componentes.

5.2 Responsabilidade de cada aplicação

Para cada aplicação, documentar:

- finalidade;
- usuários;
- funcionalidades;
- dependências;
- dados tratados;
- integrações;
- riscos;
- limitações;
- pontos de melhoria;
- partes compartilhadas.

5.3 Fluxos entre aplicações

Mapear pelo menos:

Fluxo de aquisição

Landing Page → Chrome Web Store → instalação → onboarding da extensão.

Fluxo de cadastro

Landing Page → autenticação → criação de conta → extensão → painel do usuário.

Fluxo de assinatura

Landing Page → seleção de plano → checkout → confirmação → banco → painel → extensão.

Fluxo de feature flags

Painel Administrativo → banco/API → extensão → habilitação ou bloqueio do recurso.

Fluxo de workspace

Extensão → armazenamento local ou remoto → painel → sincronização → importação/exportação.

Fluxo de integração

Extensão → autorização → API externa → Jira/Azure DevOps/GitHub → resposta → histórico.

Fluxo de suporte

Extensão ou landing → suporte → painel administrativo → acompanhamento → retorno ao usuário.

Fluxo de cancelamento

Painel do usuário → cancelamento → billing → atualização da assinatura → feature flags → extensão.

5.4 Matriz de consistência

Criar uma tabela comparando:

- recurso;
- nome na Landing Page;
- nome no Painel;
- nome na Extensão;
- plano anunciado;
- plano aplicado;
- status real;
- documentação;
- divergências encontradas.

Nenhuma funcionalidade pode ser anunciada com um nome e aparecer com outro nome completamente diferente sem motivo.

6. ANÁLISE DA LANDING PAGE

A Landing Page deve ser analisada como o principal canal de aquisição e confiança.

6.1 Auditoria da página atual

Avaliar:

- clareza do hero;
- proposta de valor;
- público-alvo;
- hierarquia da informação;
- chamadas para ação;
- navegação;
- responsividade;
- acessibilidade;
- performance;
- SEO;
- confiança;
- explicação de permissões;
- explicação de privacidade;
- apresentação dos planos;
- demonstração do produto;

- provas sociais;
- suporte;
- documentação;
- política de privacidade;
- termos;
- página de status;
- internacionalização;
- consistência visual.

Criar relatório contendo:

- problema encontrado;
- evidência;
- impacto;
- severidade;
- solução recomendada;
- esforço estimado;
- prioridade.

6.2 Nova estrutura recomendada

A página deve possuir:

Hero

Título recomendado:

Seu workspace de QA dentro do navegador

Subtítulo:

Prepare ambientes, execute testes, capture evidências e registre bugs sem alternar entre várias ferramentas.

Ações principais:

- Instalar extensão;
- Ver demonstração.

Ações secundárias:

- Explorar recursos;
- Entender permissões;
- Ver planos.

Demonstração do fluxo

Mostrar visualmente:

1. selecionar ambiente;
2. iniciar sessão;

3. executar teste;
4. capturar problema;
5. reunir contexto técnico;
6. gerar report;
7. enviar para uma integração.

Não mostrar apenas uma coleção de cards de funcionalidades.

Problemas resolvidos

Apresentar dores reais:

- alternar entre muitas ferramentas;
- perder tempo preparando dados;
- esquecer contexto do ambiente;
- enviar bugs incompletos;
- repetir preenchimentos;
- organizar várias URLs;
- capturar evidências manualmente;
- investigar requests em ferramentas separadas.

Principais áreas do produto

Apresentar por jornada:

- Preparar;
- Testar;
- Inspecionar;
- Evidenciar;
- Reportar.

Comparação

Comparar o fluxo tradicional com o fluxo usando a QA Toolbar.

Não atacar concorrentes diretamente.

Exemplo:

Processo tradicional	Com QA Toolbar
URLs em planilhas	Ambientes dentro do workspace
Credenciais em documentos	Contas de teste organizadas
Screenshot separado	Evidência integrada
Logs copiados manualmente	Contexto técnico associado
Bug escrito do zero	Report Builder
Várias extensões	Fluxo centralizado

Confiança e privacidade

Criar seção destacada:

Permissões explicadas, dados sob seu controle.

Apresentar:

- quais acessos são utilizados;
- por que são necessários;
- quando são ativados;
- onde os dados ficam;
- como revisar;
- como excluir;
- como controlar domínios;
- como proteger dados sensíveis.

Integrações

Exibir somente integrações funcionais.

Classificações permitidas:

- Disponível;
- Beta;
- Em desenvolvimento;
- Planejada.

Não apresentar integração planejada como pronta.

Planos

Apresentar planos simples e comparáveis.

Mostrar:

- público;
- recursos principais;
- limites;
- preço mensal;
- preço anual;
- economia real do anual;
- período de teste, quando existir;
- política de cancelamento;
- impostos ou moeda, quando necessário.

Documentação

Adicionar acesso a:

- primeiros passos;

- central de confiança;
- permissões;
- segurança;
- recursos;
- integrações;
- importação e exportação;
- planos;
- FAQ;
- changelog;
- suporte.

Rodapé

Incluir:

- política de privacidade;
- termos;
- contato;
- suporte;
- documentação;
- status;
- Chrome Web Store;
- versão atual;
- idiomas;
- empresa ou responsável pelo produto.

6.3 Confiança sem causar alarme

A seção de permissões não deve usar linguagem excessivamente jurídica na apresentação inicial.

Utilizar duas camadas:

Explicação simples

Exemplo:

Alguns recursos precisam interagir com a página aberta para identificar elementos, capturar evidências ou inspecionar requisições. Esses acessos são usados pelas ferramentas que você ativa.

Explicação técnica expandida

Exibir tabela:

Permissão	Recurso	Quando é usada	Controle disponível
Conteúdo da página	Captura e inspeção	Ao ativar a ferramenta	Encerrar sessão ou remover acesso
Aba atual	Evidências	Durante a captura	Permissão por aba ou domínio, quando disponível

Permissão	Recurso	Quando é usada	Controle disponível
Downloads	Exportações	Quando o usuário baixa um arquivo	Ação manual
Storage	Preferências e workspaces	Ao salvar configurações	Exportar ou excluir dados

O texto deve refletir o comportamento real.

6.4 SEO e descoberta

Implementar:

- title e description adequados;
- Open Graph;
- Twitter Cards;
- sitemap;
- robots;
- schema de software;
- FAQ estruturada;
- páginas específicas por recurso;
- páginas específicas por integração;
- páginas em PT-BR, inglês e espanhol;
- URLs canônicas;
- alt text;
- headings corretos;
- performance.

Possíveis páginas:

- /pt-br/ ;
- /en/ ;
- /es/ ;
- /features/evidence ;
- /features/workspaces ;
- /features/network ;
- /features/macros ;
- /integrations/jira ;
- /integrations/azure-devops ;
- /security ;
- /permissions ;
- /pricing ;
- /docs .

7. ANÁLISE DO PAINEL ADMINISTRATIVO

O painel não deve ser tratado apenas como uma tela interna simples.

Ele deve permitir operar o SaaS com segurança.

7.1 Identificar tipos de painel

Verificar se o projeto possui:

- painel do usuário;
- painel da organização;
- painel administrativo interno;
- painel de suporte.

Caso estejam misturados, separar responsabilidades e permissões.

7.2 Perfis e papéis

Criar ou revisar papéis como:

Usuário individual

Pode:

- gerenciar perfil;
- assinatura;
- workspaces pessoais;
- integrações pessoais;
- exportação;
- exclusão de dados.

Membro de equipe

Pode:

- acessar workspaces compartilhados;
- utilizar templates;
- executar recursos permitidos;
- visualizar dados autorizados.

Gestor da organização

Pode:

- convidar membros;
- remover membros;
- atribuir papéis;
- gerenciar assinatura;
- configurar políticas;
- controlar integrações;
- consultar uso.

Suporte

Pode:

- localizar usuários;
- consultar eventos limitados;
- analisar erros;
- registrar atendimento.

Não deve visualizar dados sensíveis de workspaces sem autorização e necessidade.

Administrador do sistema

Pode:

- gerenciar planos;
- feature flags;
- limites;
- status;
- configurações;
- incidentes;
- suporte;
- métricas.

Ações críticas devem possuir auditoria.

7.3 Dashboard operacional

Criar indicadores como:

- usuários cadastrados;
- usuários ativos;
- novas instalações;
- ativações;
- conversões;
- assinaturas;
- cancelamentos;
- trials;
- receita recorrente;
- falhas de pagamento;
- uso por funcionalidade;
- erros da extensão;
- versões instaladas;
- distribuição por navegador;
- integrações utilizadas;
- consumo de armazenamento;
- tickets de suporte.

Não usar apenas métricas de vaidade.

7.4 Métricas de produto

Acompanhar funil:

1. visitou a landing;
2. clicou para instalar;
3. instalou;
4. abriu a extensão;
5. criou ou importou workspace;
6. iniciou sessão;
7. capturou evidência;
8. gerou report;
9. conectou integração;
10. retornou ao produto;
11. iniciou trial;
12. assinou;
13. renovou;
14. cancelou.

Identificar:

- taxa de ativação;
- retenção em 1, 7 e 30 dias;
- recursos mais usados;
- abandono do onboarding;
- falhas por versão;
- conversão por plano.

7.5 Feature flags

O painel deve permitir:

- criar flag;
- editar;
- ativar;
- desativar;
- liberar por plano;
- liberar por usuário;
- liberar por organização;
- liberar por percentual;
- liberar por versão;
- definir data;
- configurar rollback;
- visualizar histórico;
- adicionar descrição;
- adicionar responsável;
- definir status beta ou experimental.

Não permitir exclusão irreversível de flags que estejam em uso.

7.6 Planos e catálogo de recursos

Criar um catálogo central de funcionalidades.

Cada recurso deve ter:

- chave;
- nome público;
- nome interno;
- descrição;
- categoria;
- status;
- planos;
- limites;
- dependências;
- versão mínima;
- disponibilidade;
- documentação;
- flag relacionada.

Landing, painel e extensão devem utilizar a mesma fonte de verdade.

7.7 Usuários

A administração deve permitir:

- pesquisar;
- filtrar;
- visualizar status;
- identificar plano;
- consultar versão da extensão;
- consultar último acesso;
- consultar organização;
- consultar falhas;
- suspender quando necessário;
- reativar;
- revogar sessões;
- redefinir recursos;
- registrar observações internas;
- exportar dados administrativos permitidos.

Não expor:

- senha;
- token completo;
- dados financeiros completos;
- credenciais salvas;
- cartões de teste;
- cookies;
- conteúdo de páginas.

7.8 Assinaturas e billing

Implementar ou revisar:

- plano;
- periodicidade;
- status;
- trial;
- início;
- renovação;
- cancelamento;
- grace period;
- falha de pagamento;
- cupom;
- histórico;
- fatura;
- upgrade;
- downgrade;
- prorrata;
- reativação;
- reembolso, quando aplicável.

A extensão deve refletir mudanças de plano com segurança e sem inconsistência.

7.9 Suporte

Criar estrutura para:

- tickets;
- categoria;
- prioridade;
- status;
- responsável;
- usuário;
- organização;
- versão;
- navegador;
- diagnóstico;
- anexos;
- histórico;
- resposta;
- resolução.

Adicionar na extensão uma opção de diagnóstico que gere dados técnicos sanitizados.

7.10 Auditoria

Registrar ações como:

- alteração de plano;

- criação de cupom;
- mudança de feature flag;
- suspensão de usuário;
- exclusão;
- alteração de limites;
- mudança de papéis;
- acesso administrativo;
- revogação de sessão;
- alteração de integração.

Cada registro deve conter:

- ator;
- ação;
- data;
- entidade;
- resultado;
- metadados seguros.

Nunca registrar tokens ou dados sensíveis em texto puro.

7.11 Segurança administrativa

Implementar ou revisar:

- RBAC;
- menor privilégio;
- expiração de sessão;
- MFA para administradores, quando disponível;
- confirmação para ações críticas;
- rate limiting;
- proteção de rotas;
- validação server-side;
- logs;
- tratamento de erro;
- proteção contra enumeração;
- políticas de senha;
- proteção de secrets.

8. ANÁLISE DA EXTENSÃO CHROME

A extensão é a aplicação principal para o usuário de QA.

8.1 Auditoria técnica

Analisar:

- Manifest V3;

- permissions;
- host_permissions;
- optional_permissions;
- optional_host_permissions;
- background service worker;
- content scripts;
- injected scripts;
- side panel;
- popup;
- options page;
- storage;
- IndexedDB;
- localStorage;
- sessionStorage;
- sincronização;
- mensagens internas;
- downloads;
- screenshots;
- gravações;
- requests;
- declarativeNetRequest;
- webRequest, caso aplicável;
- segurança de mensagens;
- CSP;
- dependências;
- bundle;
- performance;
- memória;
- vazamentos;
- listeners;
- compatibilidade;
- atualização.

8.2 Auditoria de funcionalidades

Criar tabela contendo:

- ferramenta;
- localização;
- funcionamento;
- plano;
- flag;
- permissão;
- dado acessado;
- risco;
- teste;
- documentação;
- status.

Classificar:

- estável;
- beta;
- experimental;
- incompleta;
- legada;
- duplicada;
- não utilizada.

8.3 Nova navegação da extensão

Organizar por jornada:

Início

- contexto atual;
- ambiente;
- país;
- URL;
- sessão;
- ações recentes;
- favoritos;
- status de ferramentas ativas.

Preparar

- workspaces;
- clientes;
- projetos;
- produtos;
- países;
- ambientes;
- URLs;
- contas;
- métodos de pagamento;
- dados;
- Faker Fill;
- Input Lab;
- data e horário;
- cookies e storage.

Testar

- Macro Studio;
- interações;
- viewport;
- breakpoints;
- mouse;
- teclado;
- ações rápidas;

- utilitários de página.

Inspecionar

- elementos;
- seletores;
- IDs;
- roles;
- data-test-id;
- Network Inspector;
- requests;
- responses;
- headers;
- console;
- storage;
- Key View.

Evidenciar

- screenshot;
- gravação;
- anotações;
- shapes;
- textos;
- Pass;
- Fail;
- Limitation;
- timeline;
- anexos.

Reportar

- bug;
- aprovação;
- limitação;
- impedimento;
- reteste;
- template;
- preview;
- exportação;
- integração.

Configurações

- idioma;
- tema;
- privacidade;
- permissões;
- domínios;
- integrações;
- planos;
- importação;

- exportação;
- diagnóstico;
- ajuda.

8.4 Command Palette

Criar uma busca de comandos.

Exemplos:

- iniciar sessão;
- capturar screenshot;
- gravar;
- trocar ambiente;
- abrir Network Inspector;
- limpar cache;
- preencher formulário;
- congelar horário;
- registrar bug;
- exportar workspace;
- importar workspace.

Permitir atalho configurável.

8.5 Favoritos

Permitir:

- fixar ferramentas;
- reordenar;
- ocultar;
- criar atalhos;
- salvar preferência por workspace;
- interface simplificada;
- interface completa.

8.6 Sessão de teste

Criar o conceito central de sessão.

Uma sessão deve conter:

- workspace;
- cliente;
- projeto;
- produto;
- país;
- ambiente;
- URL;
- usuário de teste;

- data inicial;
- duração;
- status;
- passos;
- ações;
- evidências;
- screenshots;
- gravações;
- requests;
- erros;
- console;
- observações;
- resultado;
- report.

Iniciar

Botão:

Iniciar sessão de teste

Utilizar o contexto atual.

Durante

Mostrar:

- cronômetro;
- contexto;
- status;
- evidências;
- erros;
- ferramentas ativas;
- botão de finalizar.

Finalizar

Gerar resumo editável:

- cenário;
- ambiente;
- duração;
- resultado;
- evidências;
- contexto técnico;
- observações;
- próximos passos.

Permitir:

- salvar;

- copiar;
- exportar;
- criar bug;
- enviar para integração;
- anexar a item existente.

8.7 Report Builder

Suportar:

- bug;
- aprovação;
- limitação;
- impedimento;
- reteste;
- melhoria;
- risco;
- sessão;
- evidência.

Campos possíveis:

- título;
- descrição;
- pré-condições;
- passos;
- esperado;
- atual;
- ambiente;
- dispositivo;
- sistema operacional;
- navegador;
- viewport;
- país;
- severidade;
- prioridade;
- labels;
- tags;
- anexos;
- logs;
- requests;
- responses;
- console;
- task;
- test case.

Permitir templates:

- globais;
- pessoais;
- por organização;

- por projeto;
- por integração.

8.8 Segurança de dados de teste

Contas e cartões cadastrados devem ser explicitamente identificados como dados de teste.

Mostrar aviso:

Cadastre somente credenciais, contas e métodos de pagamento destinados a testes. Não utilize dados pessoais, cartões reais ou credenciais particulares.

Adicionar:

- mascaramento;
- revelar sob ação;
- cópia controlada;
- timeout para ocultar;
- proteção contra inclusão em evidência;
- exclusão;
- exportação segura;
- aviso antes de compartilhar.

8.9 Redação de dados sensíveis

Detectar e ocultar, quando possível:

- password;
- Authorization;
- cookies de sessão;
- JWT;
- API keys;
- números de cartão;
- CVV;
- e-mail;
- telefone;
- documentos;
- seletores configurados;
- valores definidos pelo usuário.

Mostrar preview antes de compartilhar.

Mensagem:

Revise a evidência antes de enviar. A detecção automática reduz riscos, mas pode não identificar todos os dados sensíveis.

8.10 Ferramentas ativas

Sempre mostrar indicadores quando houver:

- gravação;
- captura de rede;
- response override;
- request override;
- horário congelado;
- macro;
- sessão;
- emulação;
- inspeção.

Nenhuma ferramenta sensível deve ficar ativa silenciosamente.

8.11 Desempenho

Analisar o tamanho da extensão.

Objetivos:

- remover arquivos desnecessários;
- remover assets duplicados;
- otimizar imagens;
- remover sourcemaps de produção;
- importar bibliotecas de forma granular;
- lazy load;
- code splitting;
- descarregar módulos inativos;
- evitar content scripts pesados;
- evitar listeners permanentes desnecessários;
- reduzir memória;
- revisar gravação;
- revisar dependências.

Meta:

- reduzir pelo menos 25% do bundle;
- buscar 40% quando tecnicamente possível;
- documentar impedimentos.

9. CENTRAL DE CONFIANÇA E PERMISSÕES

A estratégia precisa existir nos três aplicativos.

9.1 Na Landing Page

Criar:

- página `/permissions`;
- página `/security`;
- resumo simples;
- tabela detalhada;
- FAQ;
- contato;
- changelog de segurança;
- explicação local-first;
- fluxo de exclusão.

9.2 Na Extensão

Criar:

- central de permissões;
- status de cada acesso;
- funcionalidades relacionadas;
- domínios autorizados;
- permissões opcionais;
- revogação;
- exclusões;
- indicadores;
- histórico de consentimento, quando necessário;
- modo confidencial.

9.3 No Painel

Criar:

- políticas organizacionais;
- allowlist;
- blocklist;
- retenção;
- recursos permitidos;
- integrações autorizadas;
- logs;
- versões;
- controle de compartilhamento;
- opção de impedir upload automático;
- opção de proibir captura de request body;
- opção de impedir uso em domínios definidos.

9.4 Estratégia correta de comunicação

Não tratar as permissões como algo vergonhoso.

Explicar:

A extensão possui ferramentas que precisam interagir com a página aberta, como captura de evidências, inspeção de elementos e análise de requisições. Cada acesso deve estar associado a um recurso visível e controlável.

Explicar também:

O aviso apresentado pelo Chrome descreve o nível técnico máximo de acesso permitido à extensão. A QA Toolbar deve usar esse acesso somente para as ferramentas habilitadas e conforme os controles apresentados ao usuário.

Essa afirmação só pode ser usada se for verdadeira no comportamento implementado.

9.5 Permissões progressivas

Investigar:

- optional_permissions;
- optional_host_permissions;
- autorização por domínio;
- autorização por sessão;
- autorização na aba atual;
- solicitação quando o recurso for ativado;
- bloqueio de módulos sem permissão.

Não remover permissões essenciais quando isso inviabilizar o produto.

Documentar por que cada permissão permaneceu obrigatória.

10. INTEGRAÇÕES

A integração deve conectar principalmente a extensão com ferramentas de trabalho.

O painel deve gerenciar configurações, e a landing deve apresentar somente o que estiver funcional.

10.1 Arquitetura

Criar uma camada de conectores com:

- autenticação;
- OAuth;
- refresh;
- expiração;
- conexão;
- desconexão;
- projetos;
- tipos de item;

- campos;
- anexos;
- validação;
- preview;
- envio;
- tratamento de erros;
- logs sanitizados;
- status.

10.2 Jira Cloud

Primeira prioridade.

Implementar:

- autenticação;
- site;
- projeto;
- issue type;
- campos obrigatórios;
- custom fields;
- título;
- descrição;
- prioridade;
- labels;
- anexos;
- screenshots;
- gravações;
- contexto;
- issue existente;
- link da issue criada;
- mapeamento por workspace.

Não assumir schema igual em todos os projetos.

10.3 Azure DevOps

Segunda prioridade.

Implementar:

- organização;
- projeto;
- Work Item Type;
- título;
- descrição;
- Repro Steps;
- severity;
- priority;
- tags;

- Area Path;
- Iteration Path;
- custom fields;
- anexos;
- vínculo;
- link criado.

10.4 GitHub Issues

Terceira prioridade.

Implementar:

- organização;
- repositório;
- issue;
- labels;
- assignees;
- milestone;
- markdown;
- contexto;
- anexos por estratégia compatível;
- link criado.

10.5 Slack e Teams

Primeiramente oferecer:

- copiar mensagem formatada;
- copiar markdown;
- compartilhar resumo;
- abrir destino;
- webhook opcional no plano avançado.

Não exigir integração complexa se o fluxo de cópia resolver inicialmente.

10.6 Painel de integrações

O painel deve mostrar:

- integração;
- status;
- usuário;
- organização;
- data;
- último uso;
- falhas;
- permissões;
- revogação;
- logs seguros.

Tokens nunca devem aparecer completos.

10.7 Landing Page

Exibir:

- Disponível;
- Beta;
- Em desenvolvimento;
- Planejada.

Não exibir como funcional algo que ainda dependa de mock.

11. PLANOS E PREÇOS

Os planos precisam ser consistentes nas três aplicações.

Uma única fonte deve definir:

- nome;
- preço;
- periodicidade;
- recurso;
- limite;
- disponibilidade;
- trial;
- status.

11.1 Estrutura recomendada

Free — Explorar

Público:

- novos usuários;
- estudantes;
- avaliação inicial.

Incluir:

- um workspace;
- recursos básicos;
- screenshots;
- marcações;
- sessão limitada;
- report básico;
- importação;
- exportação;
- uso local;

- central de privacidade.

O plano deve permitir uma primeira vitória real.

Pro — Executar

Público:

- QA individual.

Incluir:

- workspaces adicionais;
- sessões amplas ou ilimitadas sob política justa;
- gravação;
- histórico;
- Macro Studio;
- Input Lab;
- Faker Fill avançado;
- Network Tools;
- templates;
- redação customizada;
- uma integração;
- suporte padrão.

Faixa para validação:

- R\$ 29 a R\$ 49 por mês;
- plano anual com desconto entre 15% e 25%.

Team — Colaborar

Público:

- equipes.

Incluir:

- workspaces compartilhados;
- membros;
- papéis;
- templates da equipe;
- integrações;
- biblioteca;
- políticas;
- logs;
- configurações;
- suporte prioritário.

Faixa para validação:

- R\$ 49 a R\$ 79 por usuário/mês.

Não exigir mínimo de usuários antes de existir valor colaborativo suficiente.

Business — Governar

Público:

- empresas.

Incluir futuramente:

- SSO;
- políticas administradas;
- gestão de domínios;
- retenção;
- auditoria;
- deployment;
- onboarding;
- suporte contratual;
- limites negociados;
- controles avançados.

Preço:

- sob consulta;
- ou pacote inicial transparente.

11.2 Revisar nomes existentes

Caso os planos atuais utilizem nomes como:

- Smoke Test;
- Regression Runner;
- Root Cause Analyst;
- Release Manager;

avaliar se esses nomes:

- comunicam nível comercial;
- confundem com cargos;
- dificultam comparação;
- não deixam clara a progressão.

Eles podem ser mantidos como nomes internos, modos, personas ou pacotes especiais, mas os nomes comerciais devem ser fáceis de entender.

11.3 Regras

- não cobrar para excluir dados;
- não bloquear exportação do próprio usuário;
- não anunciar “ilimitado” com limite oculto;
- mostrar política de uso justo;

- preservar assinantes atuais;
- criar grace period;
- comunicar mudanças;
- não usar dark patterns;
- não marcar “mais popular” sem estratégia real;
- mostrar limites;
- mostrar consumo;
- permitir cancelamento claro.

11.4 Sincronização entre aplicações

Quando um plano for alterado:

1. painel atualiza fonte de verdade;
 2. billing confirma;
 3. banco registra;
 4. API disponibiliza;
 5. extensão atualiza permissões de recursos;
 6. landing reflete preços públicos;
 7. histórico é preservado;
 8. usuário é comunicado quando necessário.
-

12. ONBOARDING INTEGRADO

12.1 Landing

Preparar o usuário:

- explicar resultado;
- mostrar recursos;
- explicar permissões;
- direcionar instalação.

12.2 Extensão

Primeira abertura:

1. criar workspace;
2. importar workspace;
3. explorar demonstração;
4. configurar depois.

Checklist:

- criar workspace;
- cadastrar ambiente;
- adicionar conta de teste;
- iniciar sessão;

- capturar evidência;
- gerar report;
- revisar privacidade;
- conectar integração.

12.3 Painel

Após login:

- completar perfil;
- visualizar plano;
- gerenciar organização;
- conectar integração;
- consultar uso;
- gerenciar assinatura;
- acessar documentação.

12.4 Primeira vitória

O onboarding deve conduzir a:

1. selecionar ambiente;
2. iniciar sessão;
3. capturar evidência;
4. gerar report;
5. copiar ou enviar.

Não criar tutorial longo e bloqueante.

13. ANALYTICS E PRIVACIDADE

Implementar analytics respeitando privacidade.

Eventos importantes

Landing

- view_page;
- click_install;
- click_demo;
- view_pricing;
- view_permissions;
- start_signup;
- complete_signup.

Extensão

- extension_opened;
- onboarding_started;

- onboarding_completed;
- workspace_created;
- workspace_imported;
- session_started;
- session_finished;
- screenshot_created;
- recording_created;
- report_created;
- integration_connected;
- report_sent;
- feature_used;
- permission_requested;
- permission_granted;
- permission_denied;
- error_occurred.

Painel

- plan_viewed;
- trial_started;
- subscription_started;
- subscription_changed;
- subscription_cancelled;
- member_invited;
- flag_updated;
- integration_revoked;
- support_created.

Não enviar:

- senha;
- token;
- cartão;
- conteúdo de request;
- conteúdo de response;
- conteúdo de páginas;
- dados de workspace;
- evidências;
- credenciais;
- informações sensíveis.

Utilizar IDs pseudonimizados quando possível.

Permitir desativar analytics não essenciais.

14. INTERNACIONALIZAÇÃO

Preparar:

- PT-BR;

- inglês;
- espanhol.

Centralizar traduções.

Não manter textos diretamente espalhados nos componentes.

Garantir:

- pluralização;
- datas;
- horários;
- moedas;
- preços;
- países;
- acessibilidade;
- textos longos;
- fallback;
- consistência entre aplicações.

15. DESIGN SYSTEM COMPARTILHADO

Criar ou consolidar design system compartilhado.

Definir:

- tipografia;
- spacing;
- radius;
- sombras;
- cores;
- estados;
- feedback;
- icons;
- botões;
- inputs;
- comboboxes;
- cards;
- modals;
- drawers;
- tables;
- badges;
- tooltips;
- alerts;
- empty states;
- skeletons;
- loading;
- success;
- warning;

- error;
- disabled;
- focus;
- temas claro e escuro.

A Landing pode ter apresentação comercial mais expressiva, mas deve manter identidade com painel e extensão.

O painel deve priorizar densidade e operação.

A extensão deve priorizar velocidade, clareza e pouco espaço.

16. ACESSIBILIDADE

Validar:

- navegação por teclado;
- foco visível;
- labels;
- aria;
- contraste;
- zoom;
- leitores de tela;
- mensagens de erro;
- campos obrigatórios;
- modal focus trap;
- atalhos;
- áreas clicáveis;
- tema escuro;
- redução de movimento;
- textos alternativos;
- status não dependente somente de cor.

Meta inicial:

- WCAG 2.2 AA nas interfaces principais.

17. QUALIDADE E TESTES

Criar estratégia de testes para os três aplicativos.

17.1 Landing

Testar:

- navegação;
- links;

- responsividade;
- SEO;
- formulário;
- login;
- preços;
- idiomas;
- acessibilidade;
- performance.

17.2 Painel

Testar:

- autenticação;
- autorização;
- RBAC;
- usuários;
- planos;
- assinatura;
- feature flags;
- auditoria;
- integrações;
- filtros;
- paginação;
- erros;
- exclusões;
- segurança.

17.3 Extensão

Testar:

- instalação;
- atualização;
- onboarding;
- permissões;
- workspaces;
- importação;
- exportação;
- checksum;
- storage;
- sessões;
- screenshots;
- gravações;
- macros;
- preenchimento;
- Network Inspector;
- overrides;
- evidências;
- reports;
- integrações;

- temas;
- idiomas;
- atualização de plano;
- feature flags;
- páginas SPA;
- iframes;
- múltiplas abas;
- navegação;
- perda de conexão;
- service worker suspenso;
- domínios não autorizados.

17.4 Tipos de testes

Utilizar:

- unitários;
- integração;
- E2E;
- contrato;
- regressão;
- acessibilidade;
- segurança;
- performance;
- migrations;
- compatibilidade;
- smoke tests.

18. OBSERVABILIDADE

Criar mecanismo seguro de:

- erros;
- logs;
- versão;
- ambiente;
- contexto técnico;
- performance;
- falhas de integração;
- falhas de billing;
- falhas de API;
- falhas de migrations.

Nunca registrar:

- tokens;
- senhas;
- cartões;
- cookies;

- bodies sensíveis;
- conteúdo completo da página;
- evidências privadas.

Criar correlação entre:

- Landing;
- Painel;
- API;
- Extensão;

sem expor dados privados.

19. DOCUMENTAÇÃO

Criar ou atualizar:

- README geral;
- arquitetura;
- setup;
- variáveis;
- build;
- release;
- banco;
- migrations;
- extensão;
- permissões;
- segurança;
- integrações;
- planos;
- feature flags;
- analytics;
- testes;
- troubleshooting;
- changelog;
- rollback.

Documentos mínimos:

- docs/ecosystem-audit.md ;
- docs/architecture.md ;
- docs/permissions.md ;
- docs/security.md ;
- docs/integrations.md ;
- docs/plans.md ;
- docs/analytics.md ;
- docs/release-checklist.md ;
- docs/migration-strategy.md ;
- docs/testing-strategy.md .

20. PRIORIZAÇÃO

Classificar cada melhoria com:

- impacto;
- urgência;
- esforço;
- risco;
- dependência;
- aplicação afetada;
- versão recomendada.

Utilizar:

- P0 — bloqueador;
- P1 — alta prioridade;
- P2 — importante;
- P3 — futura.

P0 sugerido

- regressões;
- exposição de dados;
- inconsistência de planos;
- permissões sem explicação;
- tokens inseguros;
- falhas de autenticação;
- importação incompatível;
- feature flags quebradas;
- landing anunciando recurso inexistente;
- painel permitindo acesso indevido;
- gravação ou inspeção ativa silenciosamente.

P1 sugerido

- onboarding;
- central de confiança;
- sessão de teste;
- Report Builder;
- catálogo central de funcionalidades;
- integração Jira;
- métricas de ativação;
- otimização do bundle;
- revisão dos planos;
- documentação.

P2 sugerido

- Azure DevOps;

- GitHub;
- colaboração;
- templates de organização;
- Slack;
- Teams;
- internacionalização completa;
- políticas empresariais.

P3 sugerido

- SSO;
- SCIM;
- TestRail;
- Zephyr;
- deployment enterprise;
- ambientes dedicados;
- integrações adicionais.

21. ORDEM DE EXECUÇÃO

Não comece alterando dezenas de arquivos.

Siga esta ordem:

Etapa 1 — Descoberta

- analisar repositório;
- mapear os três apps;
- mapear APIs;
- mapear banco;
- mapear fluxos;
- mapear permissões;
- mapear planos;
- mapear integrações;
- mapear riscos.

Etapa 2 — Relatório

Entregar:

- arquitetura;
- problemas;
- prioridades;
- proposta;
- riscos;
- migrations;
- plano de implementação.

Etapas 3 — Correções críticas

Corrigir:

- segurança;
- inconsistências;
- regressões;
- schemas;
- permissões;
- planos;
- flags;
- erros.

Etapas 4 — Fundação compartilhada

Criar:

- catálogo de recursos;
- design system;
- analytics;
- documentação;
- camada de integração;
- central de confiança.

Etapas 5 — Extensão

Implementar:

- navegação;
- sessão;
- Report Builder;
- revisão de evidências;
- indicadores;
- performance;
- onboarding.

Etapas 6 — Paineis

Implementar:

- dashboard;
- usuários;
- organizações;
- planos;
- flags;
- integrações;
- auditoria;
- suporte.

Etapas 7 — Landing

Atualizar somente depois de saber exatamente:

- o que está pronto;
- o que é beta;
- o que é planejado;
- quais planos existem;
- quais permissões são utilizadas.

Etapas 8 — Integrações

Implementar:

1. Jira;
2. Azure DevOps;
3. GitHub;
4. Slack e Teams.

Etapas 9 — Release

Executar:

- testes;
- segurança;
- performance;
- compatibilidade;
- migrations;
- rollback;
- documentação;
- changelog;
- publicação.

22. FORMATO DAS RESPOSTAS DA IA

Antes de editar o código, responda com:

1. Entendimento do projeto

Explique o que encontrou nas três aplicações.

2. Arquitetura atual

Descreva componentes e conexões.

3. Problemas encontrados

Separar por:

- Landing Page;
- Painel Administrativo;
- Extensão Chrome;
- problemas compartilhados.

4. Riscos

Classificar por severidade.

5. Proposta de arquitetura

Explicar as mudanças.

6. Backlog priorizado

Criar tabela:

ID	Aplicação	Problema	Solução	Prioridade	Esforço	Risco	Dependência
----	-----------	----------	---------	------------	---------	-------	-------------

7. Arquivos afetados

Listar antes de editar.

8. Migrations

Explicar qualquer alteração de dados.

9. Plano de testes

Explicar como validar.

10. Implementação

Executar por lotes pequenos.

Após cada lote, informar:

- arquivos alterados;
- comportamento anterior;
- comportamento novo;
- testes executados;
- resultado;
- pendências;

- riscos.

Não responder apenas que “foi implementado”.

Apresente evidências verificáveis.

23. CRITÉRIOS DE ACEITE GERAIS

A evolução será considerada bem-sucedida quando:

- as três aplicações estiverem claramente mapeadas;
- as responsabilidades estiverem separadas;
- os fluxos estiverem documentados;
- planos forem consistentes;
- funcionalidades anunciadas corresponderem ao produto real;
- permissões forem explicadas;
- dados sensíveis forem protegidos;
- usuário puder revisar evidências;
- ferramentas ativas forem visíveis;
- onboarding gerar uma primeira vitória;
- extensão possuir sessão de teste;
- Report Builder estiver integrado;
- painel permitir operação segura;
- feature flags tiverem histórico;
- analytics medir ativação;
- integrações utilizarem arquitetura extensível;
- bundle for otimizado;
- documentação estiver atualizada;
- testes cobrirem os principais fluxos;
- migrations forem seguras;
- rollback estiver documentado;
- PT-BR, inglês e espanhol estiverem preparados;
- a Landing Page transmitir valor sem promessas falsas;
- o produto estiver mais preparado para usuários individuais, equipes e empresas.

24. RESULTADO COMERCIAL ESPERADO

As mudanças devem preparar o produto para melhorar sua avaliação nos seguintes critérios:

Critério	Situação desejada
Relevância do problema	Manter alta
Qualidade da ideia	Demonstrar com fluxo completo
Diferenciação	Evidenciar workspace de QA
Apresentação comercial	Tornar clara e profissional

Critério	Situação desejada
Confiança empresarial	Melhorar com controles e documentação
Tração	Criar métricas e onboarding
Potencial SaaS	Sustentar com planos e integrações
Segurança percebida	Explicar sem alarmismo
Valor comercial	Aumentar por consistência e validação

A prioridade não deve ser adicionar o maior número possível de funcionalidades.

A prioridade deve ser:

Transformar o conjunto atual de recursos em um produto claro, confiável, integrado, mensurável e comercialmente sustentável.

INSTRUÇÃO FINAL

Comece somente pela auditoria.

Não implemente mudanças antes de apresentar:

1. mapa dos três aplicativos;
2. arquitetura;
3. fluxos;
4. problemas;
5. riscos;
6. prioridades;
7. proposta de execução;
8. migrations previstas;
9. arquivos envolvidos;
10. plano de testes.

Após a auditoria, implemente a evolução em lotes pequenos, rastreáveis e reversíveis, preservando o comportamento existente sempre que ele estiver correto.